PROJETO DE SOFTWARE

PADARIA TRIGO & MEL



01

FLUXO DA PADARIA

MenuVenda() → LerTexto() / LerInteiroPositivo() → VenderPao() → RegistrarVenda() → SalvarPaes() / SalvarVendas() → ConfirmarVenda()

CAMADA DE DADOS (STRUCTS)

02

- Define quais dados um produto ou uma venda guarda.
- Preenche os dados da struct quando ela é criada.
- Formata os dados da struct para exibir no console.

```
struct Pao
{
    public string Nome;
    public string Tipo;
    public int Quantidade;
    public decimal Preco;

    public Pao(string nome, string tipo, int quantidade, decimal preco)
    {
        Nome = nome;
        Tipo = tipo;
        Quantidade = quantidade;
        Preco = preco;
    }

    public override string ToString()
    {
        return $"Nome: {Nome} | Tipo: {Tipo} | " +
               $"Estoque: {Quantidade} | Preço: {Preco:C2}";
    }
}
```

LISTAS

Guardam os dados na memória enquanto o programa está aberto.

```
static List<Pao> paes;
static List<Doce> doces;
static List<Bebida> bebidas;
static List<Venda> vendas;
```

CAMADA DE PERSISTÊNCIA SALVAR DADOS

04

transforma cada produto em uma linha de texto separada por ; .

```
static bool SalvarPaes()
{
    List<string> linhas = new List<string>();

    foreach (Pao pao in paes)
    {
        string linha =
            LimparCampo(pao.Nome) + ";" +
            LimparCampo(pao.Tipo) + ";" +
            pao.Quantidade + ";" +
            pao.Preco.ToString(CultureInfo.InvariantCulture);

        linhas.Add(linha);
    }

    return GravarArquivo("paes.txt", linhas);
}
```

GRAVAR ARQUIVO

grava as linhas no .txt e trata erros, para o programa não fechar se a gravação falhar.

```
static bool GravarArquivo(string nomeArquivo, List<string> linhas)
{
    try
    {
        File.WriteAllLines(
            CaminhoArquivo(nomeArquivo),
            linhas
        );

        return true;
    }
    catch (Exception)
    {
        Erro(
            "Não foi possível salvar " + nomeArquivo + ". " +
            "Verifique as permissões da pasta ou se o arquivo está aberto."
        );

        return false;
    }
}
```

CARREGAR DADOS

06

Lê o arquivo ao iniciar o programa e recria os produtos na lista.

```
static List<Pao> CarregarPaes()
{
    List<Pao> lista = new List<Pao>();

    string arquivo = CaminhoArquivo("paes.txt");

    if (!File.Exists(arquivo))
    {
        return lista;
    }

    foreach (string linha in File.ReadAllLines(arquivo))
    {
        if (string.IsNullOrWhiteSpace(linha))
        {
            continue;
        }

        string[] campos = linha.Split(';');

        int quantidade;
        decimal preco;

        if (campos.Length == 4 &&
            int.TryParse(campos[2], out quantidade) &&
            quantidade >= 0 &&
            decimal.TryParse(
                campos[3],
                NumberStyles.Any,
                CultureInfo.InvariantCulture,
                out preco) &&
            PrecoValido(preco))
        {
            lista.Add(new Pao(campos[0], campos[1], quantidade, preco));
        }
        else
        {
            linhasIgnoradas.Add("paes.txt: " + linha);
        }
    }
```

CAMADA DE NEGÓCIOS
CADASTRO

07

Impede nome repetido, cria o produto e o adiciona à lista.

```
static void CadastrarPao()
{
    Console.Clear();
    Titulo("CADASTRO DE PÃO");
    Dica("Preencha os dados abaixo. Não use ponto e vírgula (;).");

    string nome = LerTexto("Nome do pão: ");

    if (PaoExiste(nome))
    {
        Erro("Já existe um pão com esse nome.");
        Pausar();
        return;
    }
```

CONSULTA

Busca o produto pelo nome, sem diferenciar maiúsculas de minúsculas.

```
static bool ConsultarPao(string nome)
{
    foreach (Pao pao in paes)
    {
        if (pao.Nome.Equals(
            nome,
            StringComparison.OrdinalIgnoreCase))
        {
            Console.WriteLine();
            Cartao(
                "PRODUTO ENCONTRADO",
                new string[] { "Nome", "Tipo", "Estoque", "Preço" },
                new string[]
                {
                    pao.Nome,
                    pao.Tipo,
                    pao.Quantidade + " un.",
                    pao.Preco.ToString("C2")
                }
            );

            return true;
        }
    }
```

EXCLUSÃO

Remove o produto da lista e atualiza o arquivo. A confirmação do usuário é verificada antes, na linha 302.

```
static bool ExcluirPao(string nome)
{
    for (int i = 0; i < paes.Count; i++)
    {
        if (paes[i].Nome.Equals(
            nome,
            StringComparison.OrdinalIgnoreCase))
        {
            paes.RemoveAt(i);

            return SalvarPaes();
        }
    }

    Erro("Pão não encontrado.");
    return false;
}
```

VENDA

Verifica o estoque, calcula o total e dá baixa na quantidade.

```
static void VenderPao(string nome, int quantidade)
{
    for (int i = 0; i < paes.Count; i++)
    {
        if (paes[i].Nome.Equals(
            nome,
            StringComparison.OrdinalIgnoreCase))
        {
            Pao pao = paes[i];

            if (quantidade > pao.Quantidade)
            {
                Erro(
                    $"Estoque insuficiente. Disponível: {pao.Quantidade}."
                );

                return;
            }

            decimal total = pao.Preco * quantidade;

            pao.Quantidade -= quantidade;
            paes[i] = pao;

            RegistrarVenda(
                pao.Nome,
                "Pão",
                quantidade,
                pao.Preco,
                total
            );

            bool salvouEstoque = SalvarPaes();
            bool salvouVendas = SalvarVendas();

            ConfirmarVenda(salvouEstoque && salvouVendas, total);

            return;
        }
    }

    Erro("Pão não encontrado.");
}
```

REGISTRO DE VENDA

Cria a venda com data e hora e a adiciona à lista de vendas.

```
static void RegistrarVenda(
    string nome,
    string categoria,
    int quantidade,
    decimal preco,
    decimal total)
{
    Venda venda = new Venda(
        nome,
        categoria,
        quantidade,
        preco,
        total,
        DateTime.Now
    );

    vendas.Add(venda);
}
```

REGISTRO DE VENDA

Cria a venda com data e hora e a adiciona à lista de vendas.

```
static void RegistrarVenda(
    string nome,
    string categoria,
    int quantidade,
    decimal preco,
    decimal total)
{
    Venda venda = new Venda(
        nome,
        categoria,
        quantidade,
        preco,
        total,
        DateTime.Now
    );

    vendas.Add(venda);
}
```

CÁLCULO DE TOTAIS

Soma as vendas por categoria e calcula o total geral.

```
static void CalcularTotaisDeVenda(
    out decimal totalPaes,
    out decimal totalDoces,
    out decimal totalBebidas,
    out decimal totalGeral)
{
    totalPaes = 0;
    totalDoces = 0;
    totalBebidas = 0;

    foreach (Venda venda in vendas)
    {
        if (venda.Categoria == "Pão")
        {
            totalPaes += venda.Total;
        }
        else if (venda.Categoria == "Doce/Bolo")
        {
            totalDoces += venda.Total;
        }
        else if (venda.Categoria == "Bebida")
        {
            totalBebidas += venda.Total;
        }
    }

    totalGeral = totalPaes + totalDoces + totalBebidas;
}
```

CAMADA DE APRESENTAÇÃO
MENU

Recebe a opção do usuário e chama a função correspondente.

```
string opcao = Console.ReadLine() ?? "";

if (opcao == "1")
{
    MenuCadastro();
}
else if (opcao == "2")
{
    MenuVenda();
}
else if (opcao == "3")
{
    MenuConsultaExclusao();
}
else if (opcao == "4")
{
    ImprimirEstoque();
    Pausar();
}
else if (opcao == "5")
{
    ImprimirRelatorio();
    Pausar();
}
else if (opcao == "0")
{
    if (SalvarTudo())
    {
        Mensagem(
            "Dados salvos. Volte sempre à Padaria Trigo & Mel!",
            ConsoleColor.Green
        );
    }
    else
    {
```

VALIDAÇÃO DE ENTRADA

Só aceita um número inteiro maior que zero. Se o valor for inválido, mostra um erro e pergunta de novo.

```
static int LerInteiroPositivo(string mensagem)
{
    while (true)
    {
        Prompt(mensagem);

        int valor;

        if (int.TryParse(Console.ReadLine(), out valor) &&
            valor > 0)
        {
            return valor;
        }

        Erro("Digite um número inteiro maior que zero.");
    }
}
```